Here is the implementation for both components:

1. **The Silero VAD Engine (VadEngine.kt)** for real-time offline interruption detection using ONNX Runtime.
2. **The Safe Room UI (RoomScreen.kt)** featuring interactive "Pass the Baton" queue states and visual feedback.

1. Silero VAD Engine (VadEngine.kt)

This class uses Microsoft's **ONNX Runtime** to run the lightweight, high-precision **Silero VAD** model locally on Android. It detects voice activity in real time without sending audio off the device.

Download the Model File

Place the pre-trained silero_vad.onnx model inside your project directory at:
app/src/main/assets/silero_vad.onnx

core/audio/VadEngine.kt

```
Kotlin
package com.mutepoint.app.core.audio

import android.content.Context
import ai.onnxruntime.OnnxTensor
import ai.onnxruntime.OrtEnvironment
import ai.onnxruntime.OrtSession
import java.nio.FloatBuffer
import javax.inject.Inject
import javax.inject.Singleton

@Singleton
class VadEngine @Inject constructor
    private val context: Context
{
    private var ortEnv: OrtEnvironment? = null
    private var ortSession: OrtSession? = null

    // Silero VAD internal recurrent states (h & c)
    private var hState: OnnxTensor? = null
    private var cState: OnnxTensor? = null

    private val sampleRate = 16000L
    private val frameSize = 512 // Process 512 samples (~32ms chunks)
```

```
    init {
        initOrtEngine()
    }

    private fun initOrtEngine() {
        runCatching {
            ortEnv = OrtEnvironment.getEnvironment()
            val modelBytes = context.assets.open("silero_vad.onnx").readBytes()
            ortSession = ortEnv?.createSession(modelBytes, OrtSession.SessionOptions())
            resetStates()
        }
    }

    /**
     * Resets hidden RNN states. Call this when starting a new meeting or stream.
     */
    fun resetStates() {
        val env = ortEnv ?: return
        hState?.close()
        cState?.close()

        // State dimensions for Silero VAD v4: [2, 1, 64]
        val shape = longArrayOf(2, 1, 64)
        val emptyBuffer = FloatArray(2 * 1 * 64) { 0f }

        hState = OnnxTensor.createTensor(env, FloatBuffer.wrap(emptyBuffer), shape)
        cState = OnnxTensor.createTensor(env, FloatBuffer.wrap(emptyBuffer), shape)
    }

    /**
     * Processes a chunk of PCM 16-bit Float Audio (16kHz) and returns speech probability [0.0 to 1.0].
     */
    fun processFrame(pcmFloatArray: FloatArray): Float {
        val env = ortEnv ?: return 0f
        val session = ortSession ?: return 0f

        if (pcmFloatArray.size != frameSize) return 0f

        val inputTensor = OnnxTensor.createTensor(
            env,
            FloatBuffer.wrap(pcmFloatArray),
```

```
            longArrayOf(1, frameSize.toLong())
        )
        val srTensor = OnnxTensor.createTensor(env, longArrayOf(sampleRate))

        val inputs = mapOf(
            "input" to inputTensor,
            "sr" to srTensor,
            "h" to hState,
            "c" to cState
        )

        return try {
            val results = session.run(inputs)

            // Extract output probability
            val outputTensor = results[0] as OnnxTensor
            val prob = outputTensor.floatBuffer.get(0)

            // Update recurrent states for continuous audio stream
            hState?.close()
            cState?.close()
            hState = results[1] as OnnxTensor
            cState = results[2] as OnnxTensor

            inputTensor.close()
            srTensor.close()

            prob
        } catch (e: Exception) {
            0f
        }
    }

    fun release() {
        hState?.close()
        cState?.close()
        ortSession?.close()
        ortEnv?.close()
    }
}
```

2. Safe Room UI (RoomScreen.kt)

This Jetpack Compose screen implements the **"Pass the Baton"** state-driven UI. Non-speakers are muted, queued users see their queue position, and the active speaker sees a circular countdown timer with animated volume waves.

presentation/saferoom/RoomScreen.kt

```
Kotlin
package com.mutepoint.app.presentation.saferoom

import androidx.compose.animation.AnimatedContent
import androidx.compose.animation.core.*
import androidx.compose.foundation.background
import androidx.compose.foundation.border
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.shape.CircleShape
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Mic
import androidx.compose.material.icons.filled.MicOff
import androidx.compose.material.icons.filled.PanTool
import androidx.compose.material.icons.filled.PanToolAlt
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.mutepoint.app.domain.model.BatonState

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun RoomScreen(
    batonState: BatonState,
    onRequestBaton: () -> Unit,
    onPassBaton: () -> Unit
) {
    Scaffold(
        topBar = {
            TopAppBar(
```

```
            title = { Text("Mute Point • Room 01", fontWeight = FontWeight.Bold) }
        )
    }
) { padding ->
    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(padding)
            .padding(24.dp),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.SpaceBetween
    ) {
        // Room Information Header
        Card(
            colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surfaceVariant),
            modifier = Modifier.fillMaxWidth()
        ) {
            Row(
                modifier = Modifier
                    .fillMaxWidth()
                    .padding(16.dp),
                horizontalArrangement = Arrangement.SpaceBetween,
                verticalAlignment = Alignment.CenterVertically
            ) {
                Column {
                    Text("Rule: Strict Single Speaker", style = MaterialTheme.typography.titleMedium)
                    Text("Intermitten Mute Enforced", style = MaterialTheme.typography.bodySmall)
                }
                Surface(
                    color = MaterialTheme.colorScheme.primaryContainer,
                    shape = CircleShape
                ) {
                    Text(
                        text = "LIVE",
                        modifier = Modifier.padding(horizontal = 12.dp, vertical = 4.dp),
                        style = MaterialTheme.typography.labelSmall,
                        color = MaterialTheme.colorScheme.onPrimaryContainer
                    )
                }
            }
        }
```

```
        // Central Speaker Visualizer & State Hub
        Box(
            contentAlignment = Alignment.Center,
            modifier = Modifier.size(280.dp)
        ) {
            when (batonState) {
                is BatonState.HoldingBaton -> {
                    ActiveSpeakerPulseRing(timeRemainingSec = batonState.timeRemainingSeconds)
                }
                is BatonState.Listening -> {
                    ListeningRing(speakerName = batonState.activeSpeakerName)
                }
                is BatonState.InQueue -> {
                    QueueRing(position = batonState.position)
                }
                is BatonState.Idle -> {
                    IdleRing()
                }
            }
        }

        // Action Controls
        Column(
            horizontalAlignment = Alignment.CenterHorizontally,
            verticalArrangement = Arrangement.spacedBy(16.dp),
            modifier = Modifier.fillMaxWidth()
        ) {
            AnimatedContent(targetState = batonState, label = "ButtonState") { state ->
                when (state) {
                    is BatonState.HoldingBaton -> {
                        Button(
                            onClick = onPassBaton,
                            colors = ButtonDefaults.buttonColors(containerColor = MaterialTheme.colorScheme.error),
                            modifier = Modifier
                                .fillMaxWidth()
                                .height(56.dp)
                        ) {
                            Icon(Icons.Default.PanTool, contentDescription = null)
                            Spacer(modifier = Modifier.width(8.dp))
                            Text("Pass the Baton Early", fontSize = 16.sp)
                        }
```

```
            is BatonState.InQueue -> {
                OutlinedButton(
                    onClick = onPassBaton, // Leaving queue
                    modifier = Modifier
                        .fillMaxWidth()
                        .height(56.dp)
                ) {
                    Text("Leave Queue (Pos #${state.position})", fontSize = 16.sp)
                }
            }
            else -> {
                Button(
                    onClick = onRequestBaton,
                    modifier = Modifier
                        .fillMaxWidth()
                        .height(56.dp)
                ) {
                    Icon(Icons.Default.PanTool, contentDescription = null)
                    Spacer(modifier = Modifier.width(8.dp))
                    Text("Raise Hand to Speak", fontSize = 16.sp)
                }
            }
        }

        Text(
            text = if (batonState is BatonState.HoldingBaton)
                "You have the floor. Microphone is ACTIVE."
            else
                "Microphone is HARD MUTED to prevent interjection.",
            style = MaterialTheme.typography.bodySmall,
            color = MaterialTheme.colorScheme.outline
        )
    }
}

@Composable
private fun ActiveSpeakerPulseRing(timeRemainingSec: Int) {
    val infiniteTransition = rememberInfiniteTransition(label = "pulse")
    val pulseScale by infiniteTransition.animateFloat(
```

```
    initialValue = 1f
    targetValue = 1.15f
    animationSpec = infiniteRepeatable(
        animation = tween(1000, easing = LinearEasing),
        repeatMode = RepeatMode.Reverse
    )
    label = "scale"
)

Box(
    contentAlignment = Alignment.Center,
    modifier = Modifier
        .size(220.dp * pulseScale)
        .background(MaterialTheme.colorScheme.primary.copy(alpha = 0.2f), CircleShape)
) {
    Box(
        contentAlignment = Alignment.Center,
        modifier = Modifier
            .size(180.dp)
            .background(MaterialTheme.colorScheme.primary, CircleShape)
    ) {
        Column(horizontalAlignment = Alignment.CenterHorizontally) {
            Icon(
                Icons.Default.Mic,
                contentDescription = null,
                tint = Color.White,
                modifier = Modifier.size(48.dp)
            )
            Spacer(modifier = Modifier.height(8.dp))
            Text(
                text = "${timeRemainingSec}s",
                color = Color.White,
                fontSize = 28.sp,
                fontWeight = FontWeight.Bold
            )
            Text("YOU ARE SPEAKING", color = Color.White.copy(alpha = 0.8f), fontSize = 10.sp)
        }
    }
}

@Composable
private fun ListeningRing(speakerName: String)
```

```
    Box(
        contentAlignment = Alignment.Center,
        modifier = Modifier
            .size(200.
            .border(4.dp, MaterialTheme.colorScheme.secondary, CircleShape)
    ) {
        Column(horizontalAlignment = Alignment.CenterHorizontally) {
            Icon(Icons.Default.MicOff, contentDescription = null, tint =
MaterialTheme.colorScheme.secondary)
            Spacer(modifier = Modifier.height(8.
            Text(speakerName, style = MaterialTheme.typography.titleMedium, fontWeight =
FontWeight.Bold)
            Text("Has the floor", style = MaterialTheme.typography.bodySmall)
        }
    }
}

@Composable
private fun QueueRing(position: Int) {
    Box(
        contentAlignment = Alignment.Center,
        modifier = Modifier
            .size(200.
            .border(4.dp, MaterialTheme.colorScheme.tertiary, CircleShape)
    ) {
        Column(horizontalAlignment = Alignment.CenterHorizontally) {
            Text("#$position", style = MaterialTheme.typography.headlineLarge, fontWeight =
FontWeight.Bold)
            Text("In Queue", style = MaterialTheme.typography.bodyMedium)
        }
    }
}

@Composable
private fun IdleRing() {
    Box(
        contentAlignment = Alignment.Center,
        modifier = Modifier
            .size(200.
            .border(2.dp, MaterialTheme.colorScheme.outlineVariant, CircleShape)
    ) {
        Text("Floor is Open", style = MaterialTheme.typography.bodyMedium, color =
MaterialTheme.colorScheme.outline)
```



Both components are now fully structured and ready for integration into your Hilt modules and Jetpack Compose navigation flow!